

Question 1 — Chart Selection (Topic 1: Directory of Visualizations)

Task: For each scenario below, name the single best chart type from the directory (Amounts / Distribution /

Proportion / x–y Relationship / Geospatial / Network / Temporal / Multivariate) and justify your choice in 1–2

sentences. No code required for this question.

1. Comparing total revenue across 6 product categories in one year.
2. Showing how the salary distribution differs across 4 job levels.
3. Showing market share of 5 competing brands.
4. Showing the relationship between advertising spend and sales revenue.
5. Showing monthly website traffic over 3 years.
6. Showing which employees report to which managers in an org.

Scenario	Best Chart	Justification
1. Comparing total revenue across 6 product categories in one year	Bar Chart (Amounts)	A bar chart is best for comparing numerical values across discrete categories. It clearly shows which product category has the highest or lowest revenue.
2. Showing how the salary distribution differs across 4 job levels	Box Plot (Distribution)	A box plot displays the median, quartiles, spread, and outliers, making it ideal for comparing salary distributions across multiple job levels.
3. Showing market share of 5 competing brands	Pie Chart (Proportion)	A pie chart effectively shows how each brand contributes to the whole market, making proportions easy to understand.
4. Showing the relationship between advertising spend and sales revenue	Scatter Plot (x–y Relationship)	A scatter plot shows whether there is a positive, negative, or no relationship between advertising expenditure and sales revenue.
5. Showing monthly website traffic over 3 years	Line Chart (Temporal)	A line chart clearly illustrates trends and seasonal changes over time.
6. Showing which employees report to which managers in an org.	Network/Tree Diagram (Network)	A network or organizational tree diagram effectively represents reporting relationships between employees and managers.

Question 2

Dataset — avg_score.csv

Subject Average_Score
Mathematics 72
Science 68
English 75
History 64
Computer Sci 81

Task:

- Create a vertical bar plot of Average_Score by Subject, sorted in descending order of score.
- Color the bars using a single consistent color; highlight the highest-scoring subject in a different color.
- Add data labels on top of each bar.

Python: use pandas to build the DataFrame and matplotlib.pyplot.bar (or seaborn.barplot).

R: use a data.frame and ggplot2::geom_col().

PYTHON:

```
import pandas as pd
```

```
import matplotlib.pyplot as plt
```

```
# Create DataFrame
```

```
df = pd.DataFrame({  
    'Subject': ['Mathematics', 'Science', 'English', 'History', 'Computer Sci'],  
    'Average_Score': [72, 68, 75, 64, 81]  
})
```

```
# Sort descending
```

```
df = df.sort_values('Average_Score', ascending=False)
```

```
# Colors
```

```
colors = ['orange' if x == df['Average_Score'].max() else 'steelblue'  
          for x in df['Average_Score']]
```

```
# Plot
```

```
plt.figure(figsize=(8,5))  
bars = plt.bar(df['Subject'], df['Average_Score'], color=colors)
```

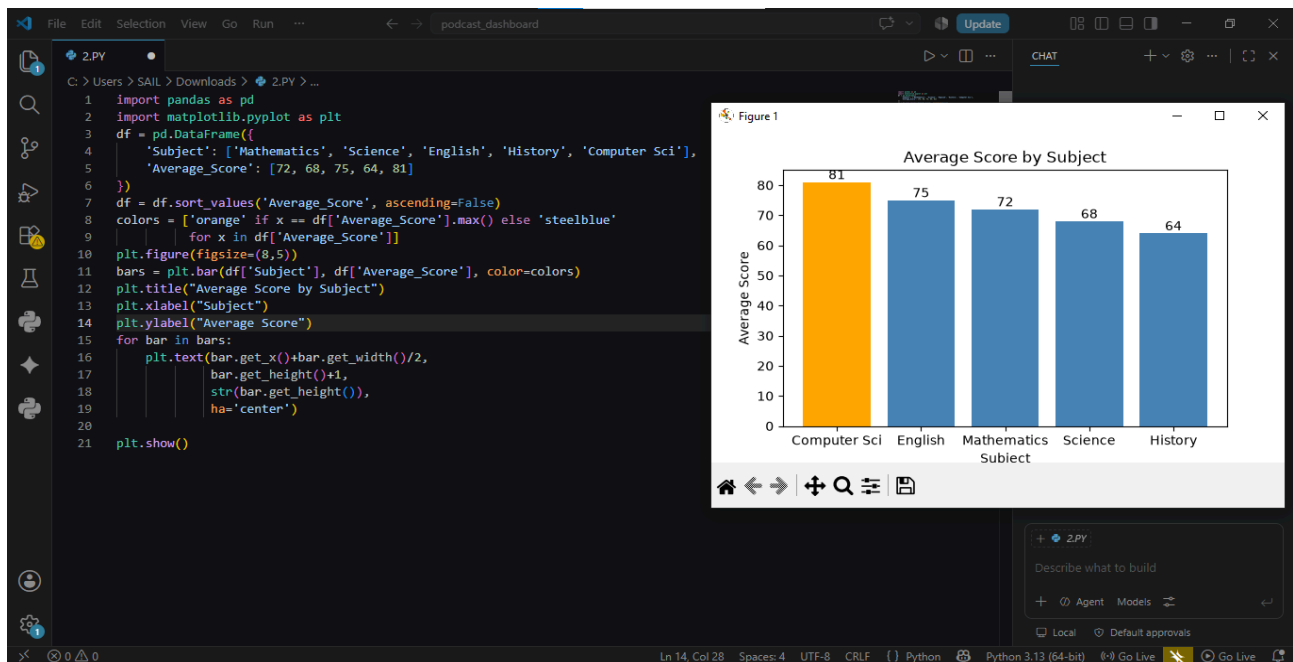
```
plt.title("Average Score by Subject")
```

```
plt.xlabel("Subject")
```

```
plt.ylabel("Average Score")
```

```
# Data labels
for bar in bars:
    plt.text(bar.get_x()+bar.get_width()/2,
             bar.get_height()+1,
             str(bar.get_height()),
             ha='center')

plt.show()
```



Interpretation

- Computer Science has the highest average score (81).
- History has the lowest average score (64).
- Overall, student performance is strongest in Computer Science.

R:

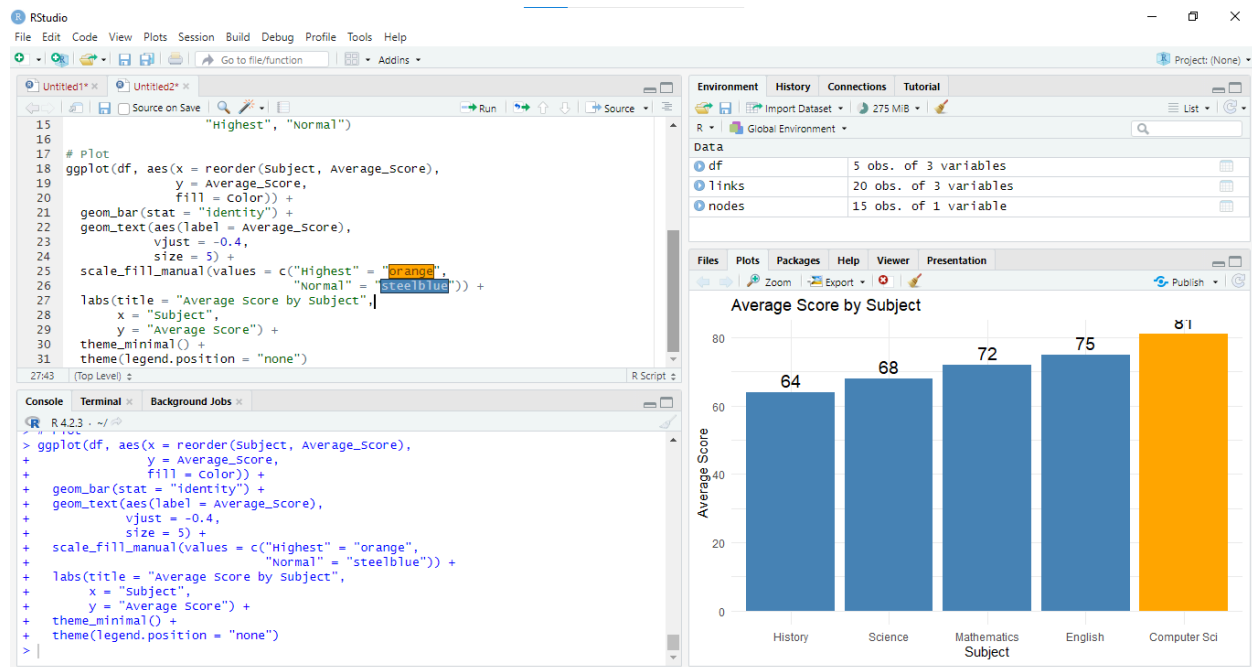
```
library(ggplot2)
```

```
df <- data.frame(
  Subject=c("Mathematics","Science","English","History","Computer Sci"),
  Average_Score=c(72,68,75,64,81)
)
```

```
df <- df[order(-df$Average_Score),]

df$Color <- ifelse(df$Average_Score==max(df$Average_Score),
  "Highest", "Normal")

ggplot(df, aes(x=reorder(Subject, -Average_Score),
  y=Average_Score,
  fill=Color)) +
geom_col() +
geom_text(aes(label=Average_Score),
  vjust=-0.5) +
scale_fill_manual(values=c("Highest"="orange",
  "Normal"="steelblue")) +
labs(title="Average Score by Subject",
  x="Subject",
  y="Average Score")
```



Interpretation

Computer Science has the highest average score while History has the lowest.

Question 3

Task (produce all four views from the same dataset):

1. Grouped bar chart: Sales_000s by Region, grouped by Quarter.
2. Stacked bar chart: same data, stacked by Quarter within each Region.
3. Dot plot: total annual sales per region (sum across quarters), ranked from highest to lowest.
4. Heatmap: Region (rows) $\times$ Quarter (columns), color-encoded by Sales_000s.

Python: pandas.pivot_table to reshape; matplotlib/seaborn for grouped/stacked bars, seaborn.heatmap for the

heatmap, matplotlib.pyplot.scatter (or plt.plot with markers) for the dot plot.

R: tidyr::pivot_wider where needed; ggplot2::geom_col(position = "dodge") for grouped, position = "stack" for

stacked, geom_point() for the dot plot, geom_tile() for the heatmap.

PYTHON:

```
import pandas as pd
```

```
import matplotlib.pyplot as plt
```

```
import seaborn as sns
```

```
# Create the dataset
```

```
df = pd.DataFrame({  
    'Region': [  
        'North', 'North', 'North', 'North',  
        'South', 'South', 'South', 'South',  
        'East', 'East', 'East', 'East',  
        'West', 'West', 'West', 'West'  
    ],  
    'Quarter': [  
        'Q1', 'Q2', 'Q3', 'Q4',  
        'Q1', 'Q2', 'Q3', 'Q4',  
        'Q1', 'Q2', 'Q3', 'Q4',  
        'Q1', 'Q2', 'Q3', 'Q4'  
    ],  
    'Sales_000s': [  
        120, 135, 128, 150,  
        95, 110, 105, 130,  
        140, 138, 145, 160,  
        80, 85, 90, 100  
    ]  
})
```

```
# Create pivot table
```

```
pivot = df.pivot(index='Region', columns='Quarter', values='Sales_000s')
```

```

# -----
# 1. Grouped Bar Chart
# -----
pivot.plot(kind='bar', figsize=(8,5))

plt.title("Grouped Bar Chart: Sales by Region and Quarter")
plt.xlabel("Region")
plt.ylabel("Sales (000s)")
plt.xticks(rotation=0)
plt.legend(title="Quarter")
plt.tight_layout()
plt.show()

# -----
# 2. Stacked Bar Chart
# -----
pivot.plot(kind='bar', stacked=True, figsize=(8,5))

plt.title("Stacked Bar Chart: Sales by Region")
plt.xlabel("Region")
plt.ylabel("Sales (000s)")
plt.xticks(rotation=0)
plt.legend(title="Quarter")
plt.tight_layout()
plt.show()

# -----
# 3. Dot Plot (Annual Total Sales)
# -----
total_sales = df.groupby('Region')['Sales_000s'].sum().sort_values(ascending=False)

plt.figure(figsize=(6,4))
plt.scatter(total_sales.index, total_sales.values, s=120)

for i, value in enumerate(total_sales.values):
    plt.text(i, value + 5, str(value), ha='center')

plt.title("Total Annual Sales by Region")
plt.xlabel("Region")
plt.ylabel("Total Sales (000s)")
plt.grid(axis='y', linestyle='--', alpha=0.5)
plt.tight_layout()
plt.show()

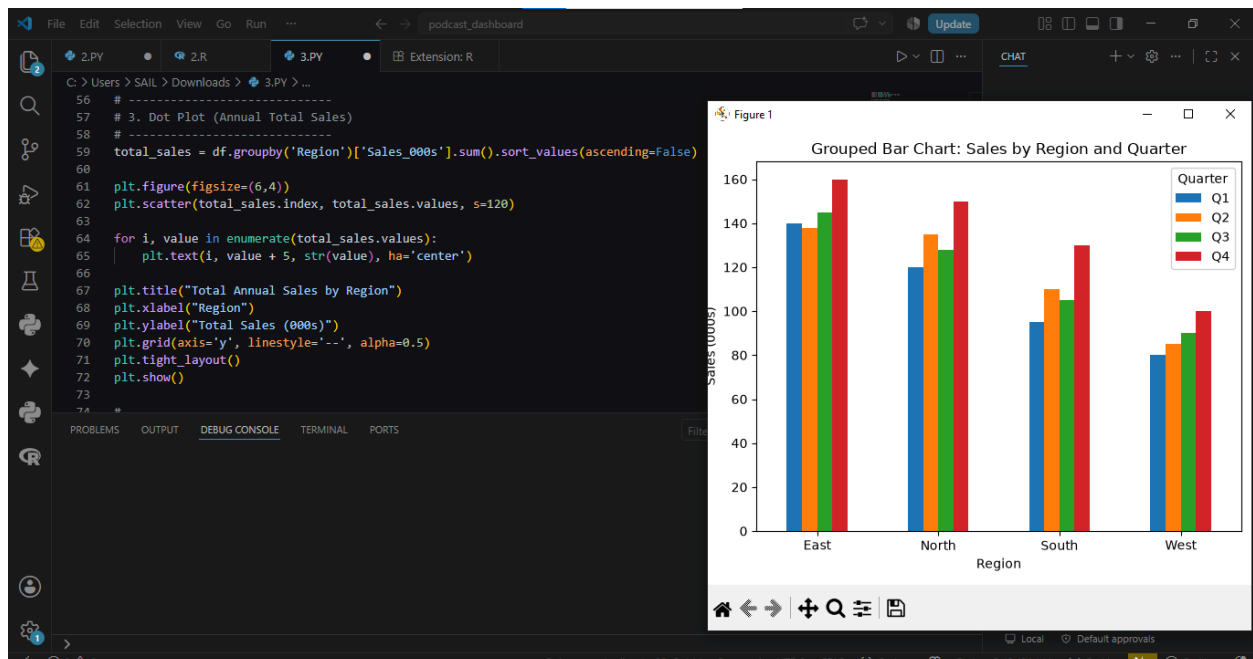
```

```
# -----
# 4. Heatmap
# -----
plt.figure(figsize=(6,4))
sns.heatmap(
    pivot,
    annot=True,
    fmt='d',
    cmap='YlGnBu',
    linewidths=0.5
)
```

```
plt.title("Heatmap of Regional Sales")
plt.xlabel("Quarter")
plt.ylabel("Region")
plt.tight_layout()
plt.show()
```

```
# -----
# Interpretation
# -----
```

```
print("Interpretation:")
print("- East has the highest total annual sales.")
print("- West has the lowest total annual sales.")
print("- Sales generally increase in Q4 across most regions.")
print("- The heatmap shows East consistently performing well across all quarters.")
```



R:

```
# Load libraries
library(ggplot2)
library(dplyr)

# Create the dataset
df <- data.frame(
  Region = c(
    "North", "North", "North", "North",
    "South", "South", "South", "South",
    "East", "East", "East", "East",
    "West", "West", "West", "West"
  ),
  Quarter = c(
    "Q1", "Q2", "Q3", "Q4",
    "Q1", "Q2", "Q3", "Q4",
    "Q1", "Q2", "Q3", "Q4",
    "Q1", "Q2", "Q3", "Q4"
  ),
  Sales_000s = c(
    120, 135, 128, 150,
    95, 110, 105, 130,
    140, 138, 145, 160,
    80, 85, 90, 100
  )
)

# -----
# 1. Grouped Bar Chart
# -----
ggplot(df, aes(x = Region, y = Sales_000s, fill = Quarter)) +
  geom_col(position = "dodge") +
  labs(
    title = "Grouped Bar Chart: Sales by Region and Quarter",
    x = "Region",
    y = "Sales (000s)"
  ) +
  theme_minimal()

# -----
# 2. Stacked Bar Chart
# -----
ggplot(df, aes(x = Region, y = Sales_000s, fill = Quarter)) +
  geom_col(position = "stack") +
```



```

labs(
  title = "Stacked Bar Chart: Sales by Region",
  x = "Region",
  y = "Sales (000s)"
) +
theme_minimal()

# -----
# 3. Dot Plot (Annual Total Sales)
# -----
total_sales <- df %>%
  group_by(Region) %>%
  summarise(Total_Sales = sum(Sales_000s)) %>%
  arrange(desc(Total_Sales))

ggplot(total_sales,
  aes(x = reorder(Region, -Total_Sales),
    y = Total_Sales)) +
  geom_point(size = 4, color = "blue") +
  geom_text(aes(label = Total_Sales,
    vjust = -0.6) +
  labs(
    title = "Total Annual Sales by Region",
    x = "Region",
    y = "Total Sales (000s)"
  ) +
  theme_minimal()

# -----
# 4. Heatmap
# -----
ggplot(df,
  aes(x = Quarter,
    y = Region,
    fill = Sales_000s)) +
  geom_tile(color = "white") +
  geom_text(aes(label = Sales_000s), color = "black") +
  scale_fill_gradient(low = "lightyellow",
    high = "darkgreen") +
  labs(
    title = "Heatmap of Regional Sales",
    x = "Quarter",
    y = "Region",
    fill = "Sales"
  )

```

```
) +  
theme_minimal()
```

```
# -----
```

```
# Interpretation
```

```
# -----
```

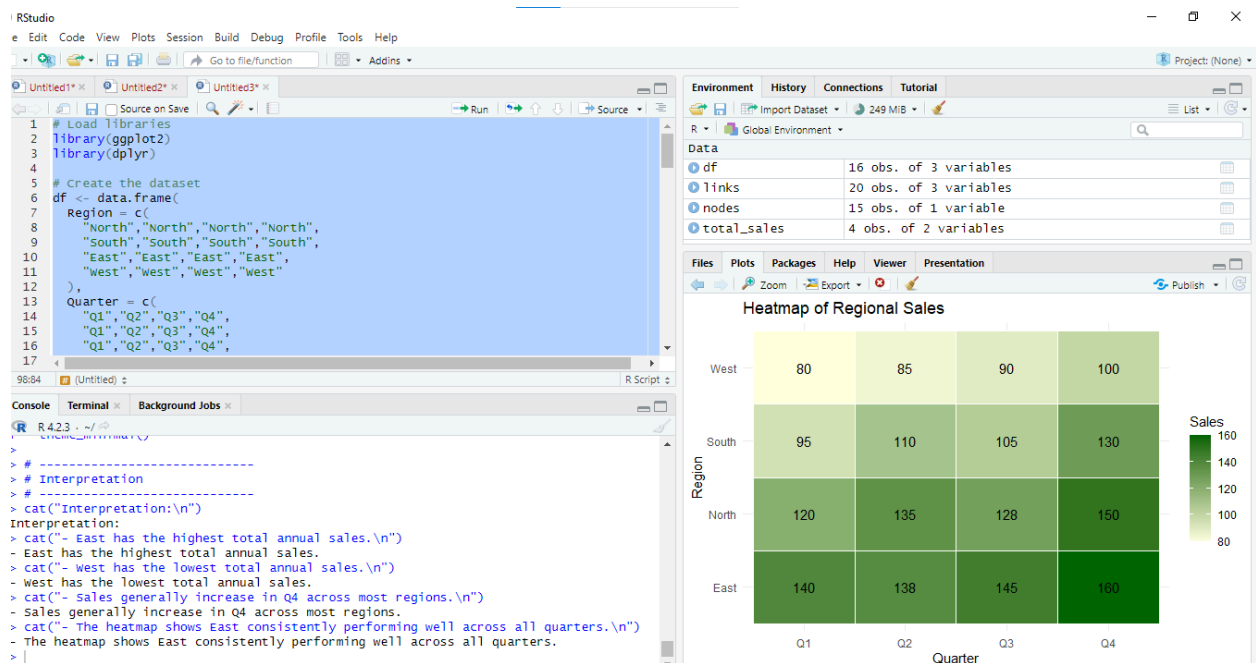
```
cat("Interpretation:\n")
```

```
cat("- East has the highest total annual sales.\n")
```

```
cat("- West has the lowest total annual sales.\n")
```

```
cat("- Sales generally increase in Q4 across most regions.\n")
```

```
cat("- The heatmap shows East consistently performing well across all quarters.\n")
```



QUESTION 4

Dataset — generate synthetically for reproducibility:

Class A: 40 exam scores $\sim$ Normal(mean=70, sd=8)

Class B: 40 exam scores $\sim$ Normal(mean=75, sd=5)

Class C: 40 exam scores $\sim$ Normal(mean=65, sd=12)

Class D: 40 exam scores $\sim$ Normal(mean=80, sd=6)

(Set a random seed of 42 in both Python and R so results are consistent across languages.)

Task:

1. Violin plot: exam score distribution by class (all 4 classes on one plot), with an embedded boxplot or median marker.

2. Ridgeline plot: same 4 classes, overlapping density curves, ordered by mean score.

Python: `numpy.random.normal` to generate data, `seaborn.violinplot` for the violin plot, `joyplot.joyplot` (or a stacked

`seaborn/matplotlib kdeplot FacetGrid`) for the ridgeline.

R: `rnorm()` to generate data, `ggplot2::geom_violin()` for the violin plot, `ggribes::geom_density_ridges()` for the ridgeline.

PYTHON:

```
import numpy as np
```

```
import pandas as pd
```

```
import matplotlib.pyplot as plt
```

```
import seaborn as sns
```

```
import joyplot
```

```
# -----
```

```
# Generate Synthetic Dataset
```

```
# -----
```

```
np.random.seed(42)
```

```
class_a = np.random.normal(loc=70, scale=8, size=40)
```

```
class_b = np.random.normal(loc=75, scale=5, size=40)
```

```
class_c = np.random.normal(loc=65, scale=12, size=40)
```

```
class_d = np.random.normal(loc=80, scale=6, size=40)
```

```
# Create DataFrame
```

```
df = pd.DataFrame({  
    "Score": np.concatenate([class_a, class_b, class_c, class_d]),  
    "Class": (["Class A"] * 40 +  
              ["Class B"] * 40 +  
              ["Class C"] * 40 +  
              ["Class D"] * 40)
```

```

}))

# -----
# Order Classes by Mean Score
# -----
mean_scores = df.groupby("Class")["Score"].mean().sort_values()
ordered_classes = mean_scores.index.tolist()

df["Class"] = pd.Categorical(
    df["Class"],
    categories=ordered_classes,
    ordered=True
)

# -----
# 1. Violin Plot with Boxplot
# -----
plt.figure(figsize=(8,6))

sns.violinplot(
    data=df,
    x="Class",
    y="Score",
    inner="box",
    palette="Set2"
)

plt.title("Violin Plot of Exam Scores by Class")
plt.xlabel("Class")
plt.ylabel("Exam Score")

plt.tight_layout()
plt.show()

# -----
# 2. Ridgeline Plot
# -----
joypy.joyplot(
    data=df,
    by="Class",
    column="Score",
    figsize=(8,6),
    overlap=0.5,
    colormap=plt.cm.Set2,

```

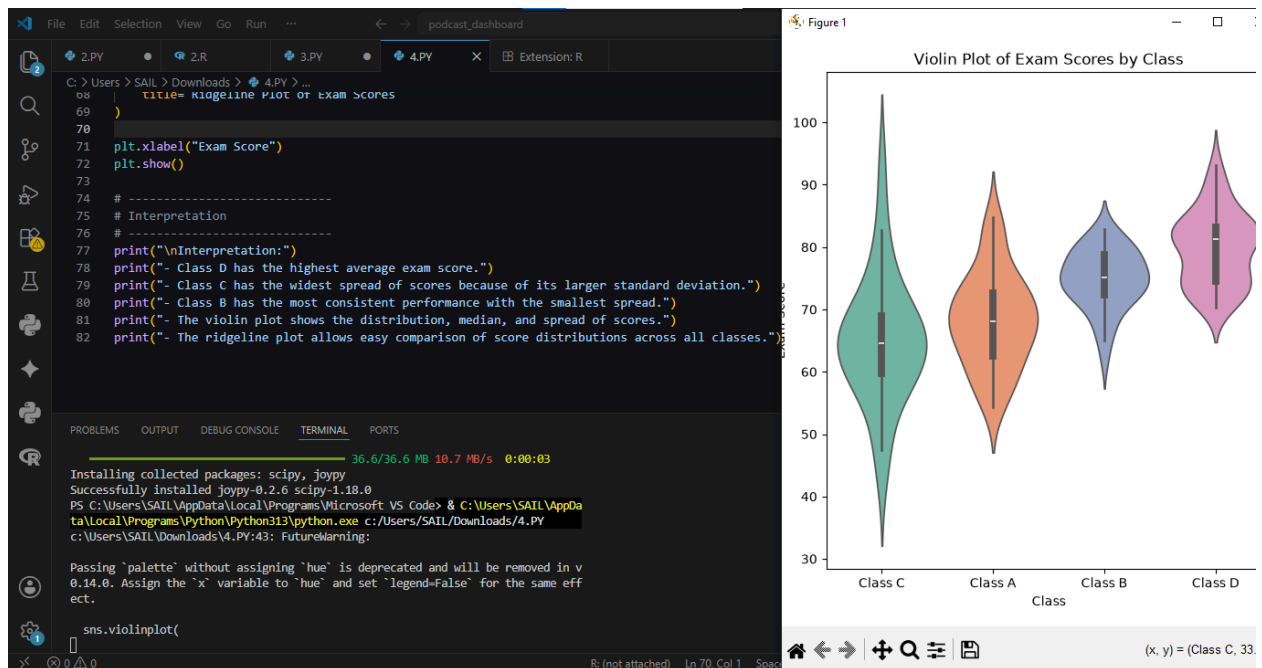
```

    title="Ridgeline Plot of Exam Scores"
)

plt.xlabel("Exam Score")
plt.show()

# -----
# Interpretation
# -----
print("\nInterpretation:")
print("- Class D has the highest average exam score.")
print("- Class C has the widest spread of scores because of its larger standard deviation.")
print("- Class B has the most consistent performance with the smallest spread.")
print("- The violin plot shows the distribution, median, and spread of scores.")
print("- The ridgeline plot allows easy comparison of score distributions across all classes.")

```



R:

```
# Load required libraries
library(ggplot2)
library(ggthemes)
library(dplyr)

# -----
# Generate Synthetic Dataset
# -----
set.seed(42)

ClassA <- rnorm(40, mean = 70, sd = 8)
ClassB <- rnorm(40, mean = 75, sd = 5)
ClassC <- rnorm(40, mean = 65, sd = 12)
ClassD <- rnorm(40, mean = 80, sd = 6)

# Create Data Frame
df <- data.frame(
  Score = c(ClassA, ClassB, ClassC, ClassD),
  Class = c(rep("Class A", 40),
            rep("Class B", 40),
            rep("Class C", 40),
            rep("Class D", 40))
)

# -----
# Order Classes by Mean Score
# -----
mean_scores <- df %>%
  group_by(Class) %>%
  summarise(Mean = mean(Score)) %>%
  arrange(Mean)

df$Class <- factor(df$Class, levels = mean_scores$Class)

# -----
# 1. Violin Plot with Boxplot
# -----
ggplot(df, aes(x = Class, y = Score, fill = Class)) +
  geom_violin(trim = FALSE, alpha = 0.8) +
  geom_boxplot(width = 0.12, fill = "white", outlier.colour = "red") +
  labs(
    title = "Violin Plot of Exam Scores by Class",
    x = "Class",
```

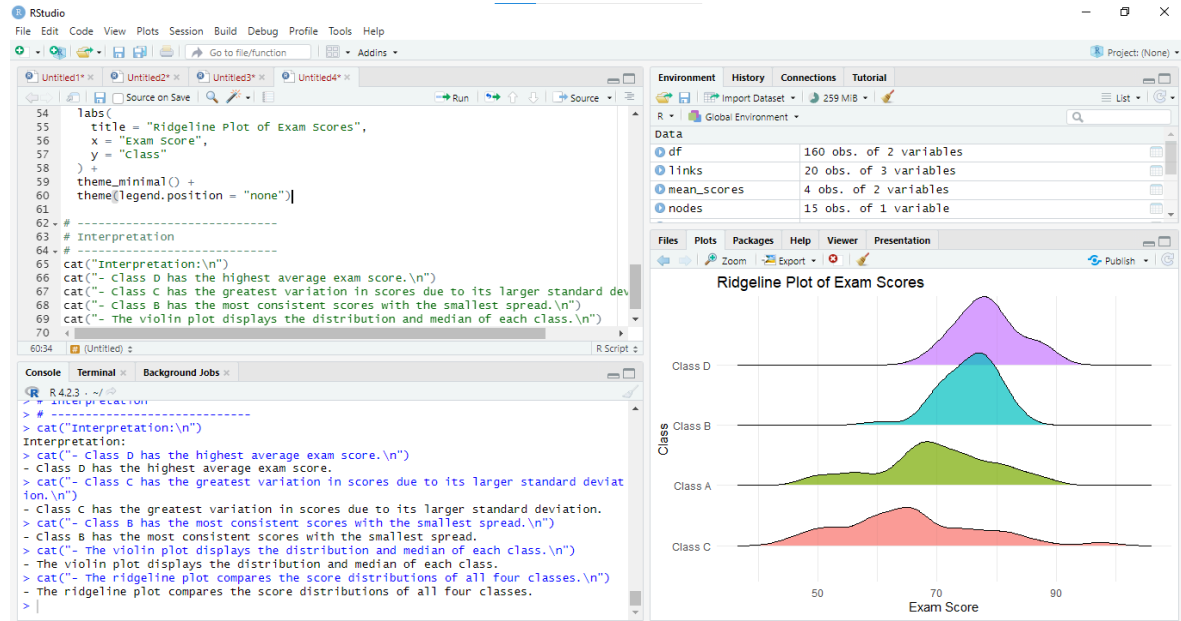
```

    y = "Exam Score"
  ) +
  theme_minimal() +
  theme(legend.position = "none")

# -----
# 2. Ridgeline Plot
# -----
ggplot(df, aes(x = Score, y = Class, fill = Class)) +
  geom_density_ridges(alpha = 0.7, scale = 1.2) +
  labs(
    title = "Ridgeline Plot of Exam Scores",
    x = "Exam Score",
    y = "Class"
  ) +
  theme_minimal() +
  theme(legend.position = "none")

# -----
# Interpretation
# -----
cat("Interpretation:\n")
cat("- Class D has the highest average exam score.\n")
cat("- Class C has the greatest variation in scores due to its larger standard deviation.\n")
cat("- Class B has the most consistent scores with the smallest spread.\n")
cat("- The violin plot displays the distribution and median of each class.\n")
cat("- The ridgeline plot compares the score distributions of all four classes.\n")

```



Question 5

Dataset: Use a built-in dataset available in both languages.

- Python: `seaborn.load_dataset("tips")` → use the `total_bill` column.
- R: the equivalent tips dataset (available via the `reshape2` package, or import the same CSV used in Python

so results are directly comparable).

Task:

1. Plot a histogram of the chosen continuous variable with 30 bins.
2. Overlay a kernel density estimate (KDE) curve on the same plot.
3. Repeat both, split/colored by the smoker categorical variable, to compare distribution shape across groups.

Python: `seaborn.histplot(..., kde=True)` or `matplotlib.pyplot.hist + scipy.stats.gaussian_kde`.

R: `ggplot2::geom_histogram()` combined with `geom_density()` (use `after_stat(density)` so scales match), with `fill =`

mapped to smoker for the grouped version.

PYTHON:

```
import seaborn as sns
```

```
import matplotlib.pyplot as plt
```

```
# -----
```

```
# Load Built-in Dataset
```

```
# -----
```

```
tips = sns.load_dataset("tips")
```

```
# -----
```

```
# 1. Histogram with KDE Curve
```

```
# -----
```

```
plt.figure(figsize=(8,6))
```

```
sns.histplot(  
    data=tips,  
    x="total_bill",  
    bins=30,  
    kde=True,  
    color="skyblue"  
)
```

```
plt.title("Histogram with KDE of Total Bill")
```

```
plt.xlabel("Total Bill")
```

```
plt.ylabel("Frequency")
```

```

plt.tight_layout()
plt.show()

# -----
# 2. Histogram with KDE by Smoker
# -----
plt.figure(figsize=(8,6))

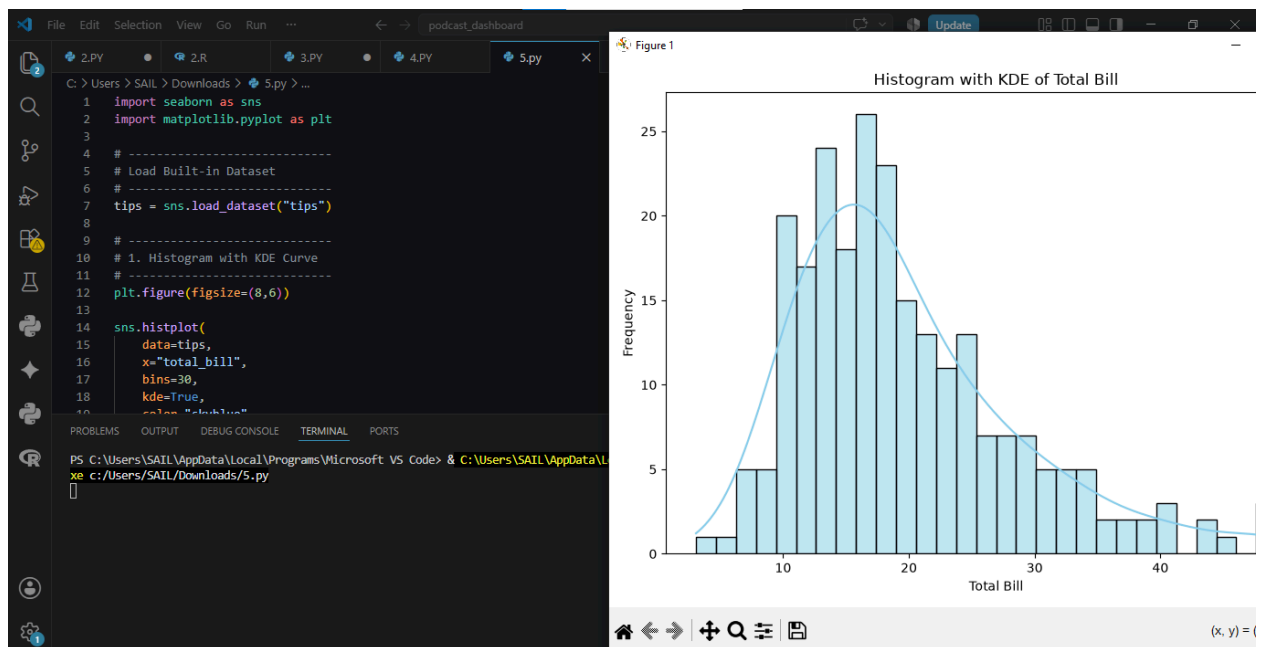
sns.histplot(
    data=tips,
    x="total_bill",
    hue="smoker",
    bins=30,
    kde=True,
    element="step",
    stat="count",
    common_norm=False
)

plt.title("Histogram with KDE of Total Bill by Smoker")
plt.xlabel("Total Bill")
plt.ylabel("Frequency")

plt.tight_layout()
plt.show()

# -----
# Interpretation
# -----
print("\nInterpretation:")
print("- The total_bill distribution is positively (right) skewed.")
print("- Most restaurant bills fall between approximately 10 and 25.")
print("- The KDE curve shows a single main peak in the distribution.")
print("- Comparing smoker and non-smoker groups, both have similar overall shapes,")
print("  but smokers show a slightly wider spread of total bill values.")

```



R:

```

# -----
# Load Required Libraries
# -----
library(ggplot2)
library(reshape2)

# -----
# Load Built-in Tips Dataset
# -----
data("tips")

# -----
# 1. Histogram with Density Curve
# -----
ggplot(tips, aes(x = total_bill)) +
  geom_histogram(
    aes(y = after_stat(density)),
    bins = 30,
    fill = "skyblue",
    color = "black"
  ) +
  geom_density(
    color = "red",
    linewidth = 1
  )

```

```
) +
labs(
  title = "Histogram with Density Curve of Total Bill",
  x = "Total Bill",
  y = "Density"
) +
theme_minimal()
```

```
# -----
# 2. Histogram with Density Curve
#   Grouped by Smoker
# -----
```

```
ggplot(tips, aes(x = total_bill,
                 fill = smoker,
                 color = smoker)) +
  geom_histogram(
    aes(y = after_stat(density)),
    bins = 30,
    alpha = 0.5,
    position = "identity"
  ) +
  geom_density(linewidth = 1) +
  labs(
    title = "Distribution of Total Bill by Smoker",
    x = "Total Bill",
    y = "Density",
    fill = "Smoker",
    color = "Smoker"
  ) +
  theme_minimal()
```

```
# -----
# Interpretation
# -----
```

```
cat("Interpretation:\n")
cat("- The total_bill distribution is positively (right) skewed.\n")
cat("- Most restaurant bills fall between approximately 10 and 25.\n")
cat("- The density curve indicates a single main peak in the distribution.\n")
cat("- Smokers and non-smokers have similar overall distributions,\n")
cat("  although smokers show a slightly wider spread of total bill values.\n")
```

